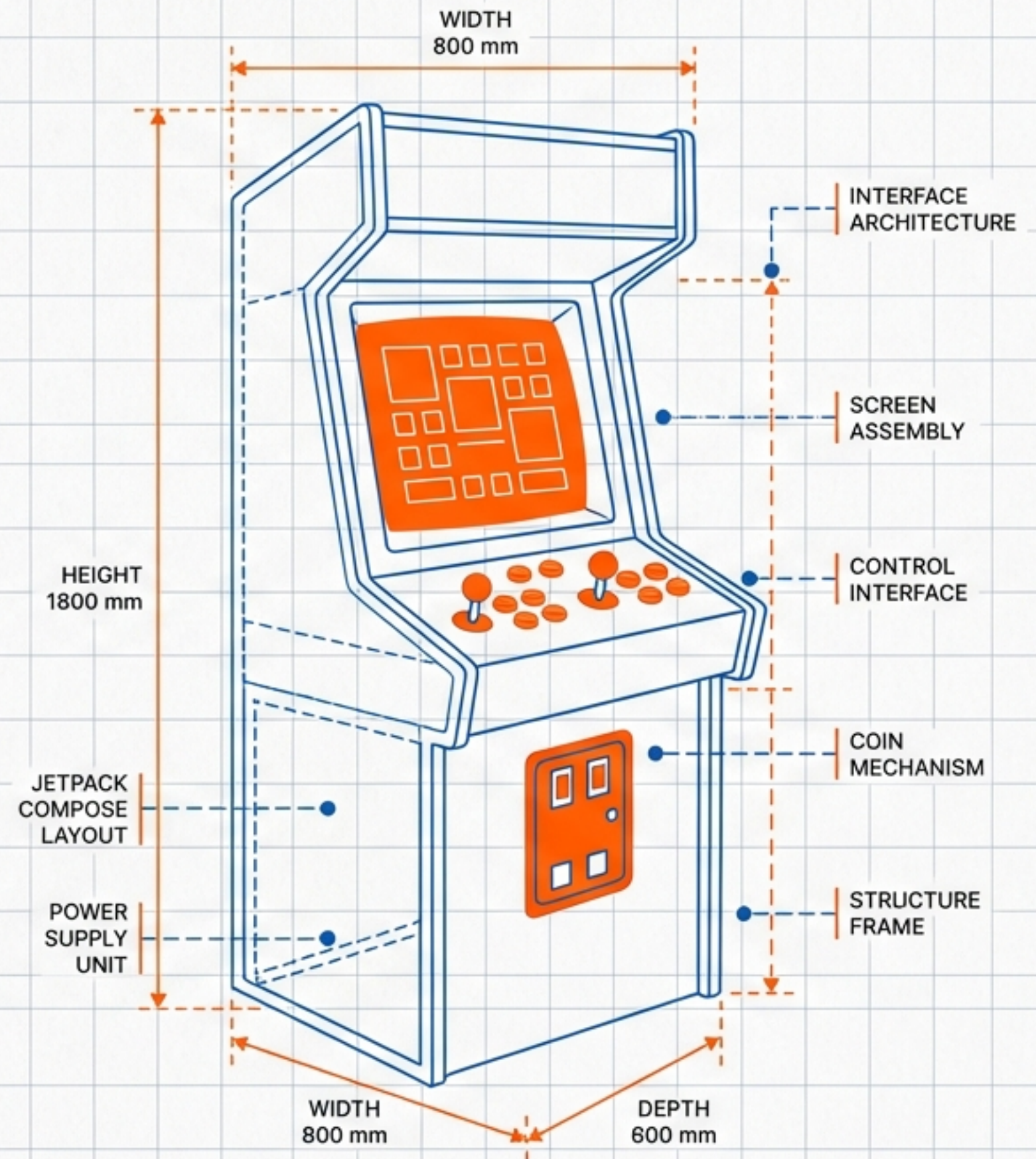


Construyendo un Arcade en Android

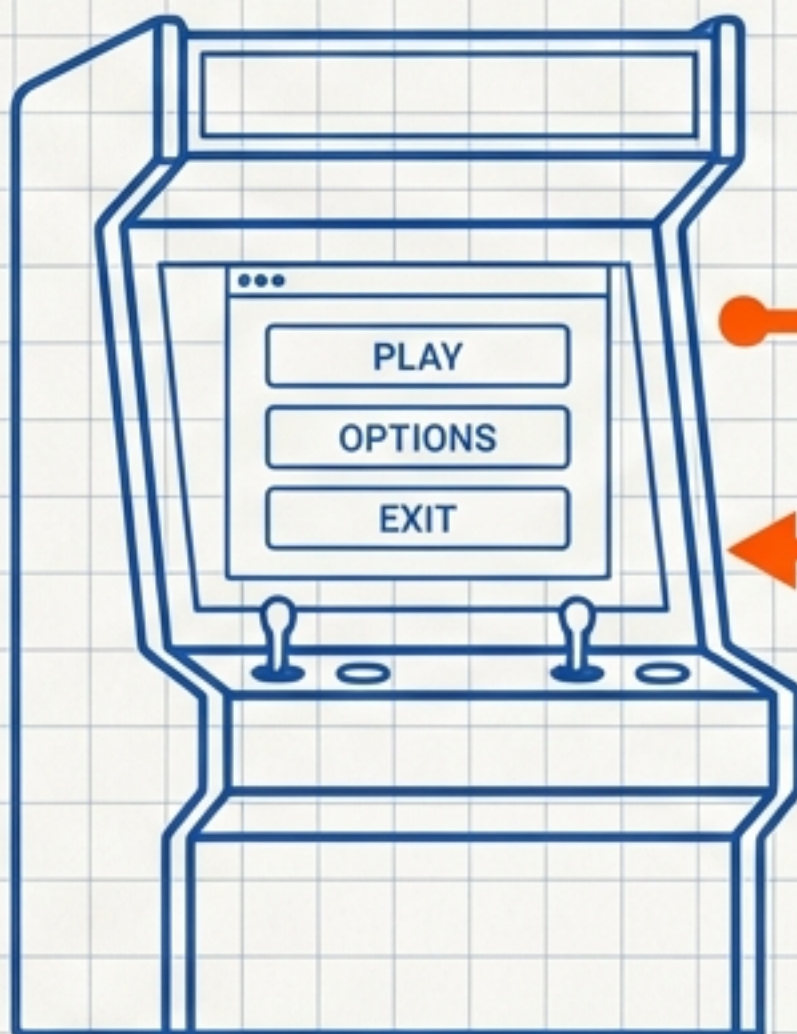
Guía visual para principiantes sobre Jetpack Compose, Kotlin y la arquitectura de interfaces.



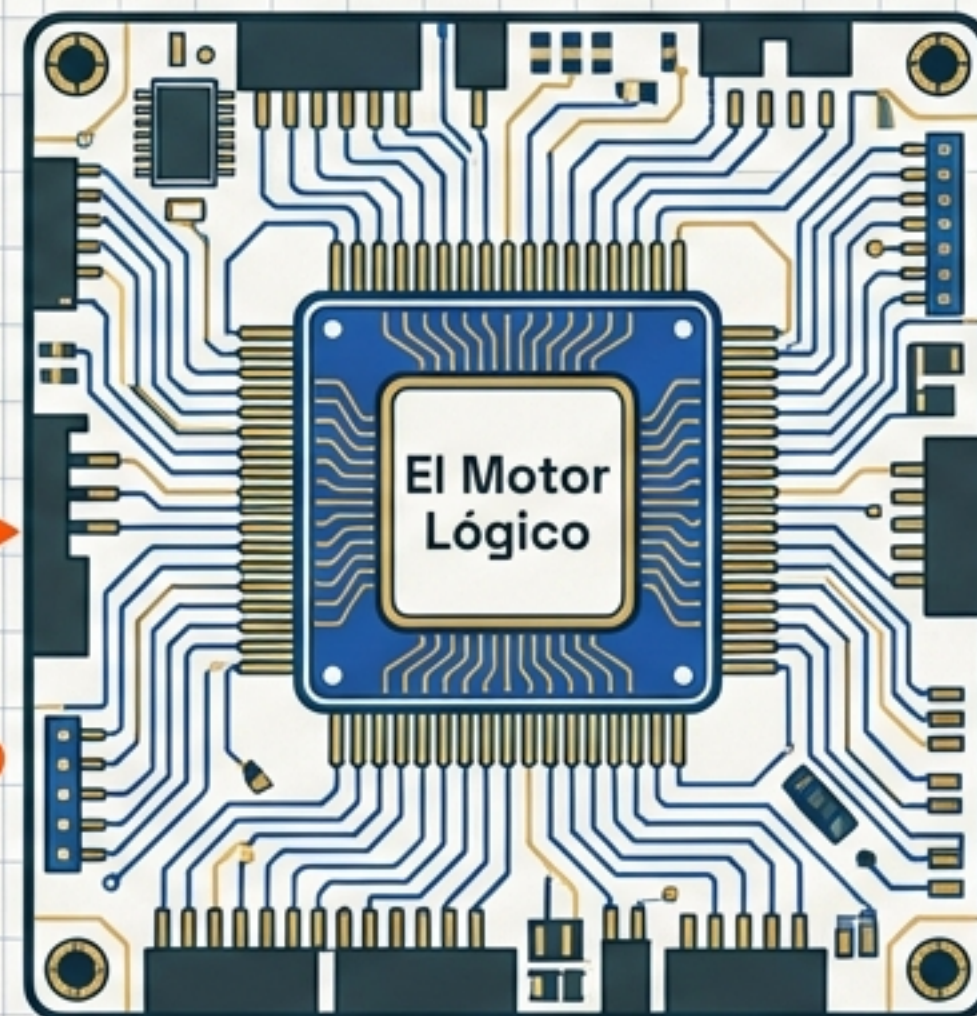
La anatomía de nuestra aplicación Mochis

Para construir una app escalable, dividimos el sistema en dos mundos que no se mezclan. Compose solo dibuja lo que la Lógica le dicta, y la Lógica no sabe nada sobre colores o píxeles.

El Mundo Visual (Jetpack Compose)



El Motor Lógico (Clases Puras de Kotlin)



Estado y Eventos

El punto de entrada: MainActivity

El puente: Conecta el sistema operativo Android con nuestro mundo de Compose

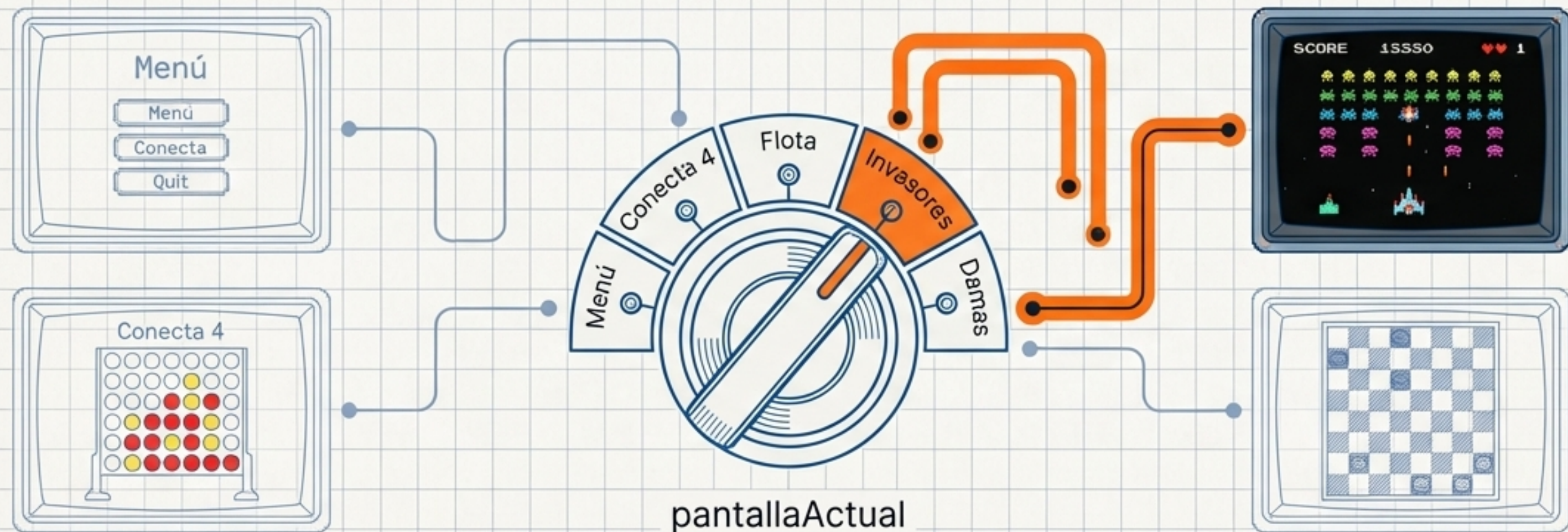
Versión simplificada: Hemos eliminado capas sobrantes para mostrar la conexión más pura.



```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent { AppMochisUnificada() }  
    }  
}
```

La magia del 'Estado' en Compose

En la programación declarativa, no abrimos ni cerramos pantallas. Cambiamos una variable de estado. Cuando el estado cambia, Compose destruye la interfaz anterior y reconstruye instantáneamente la nueva.



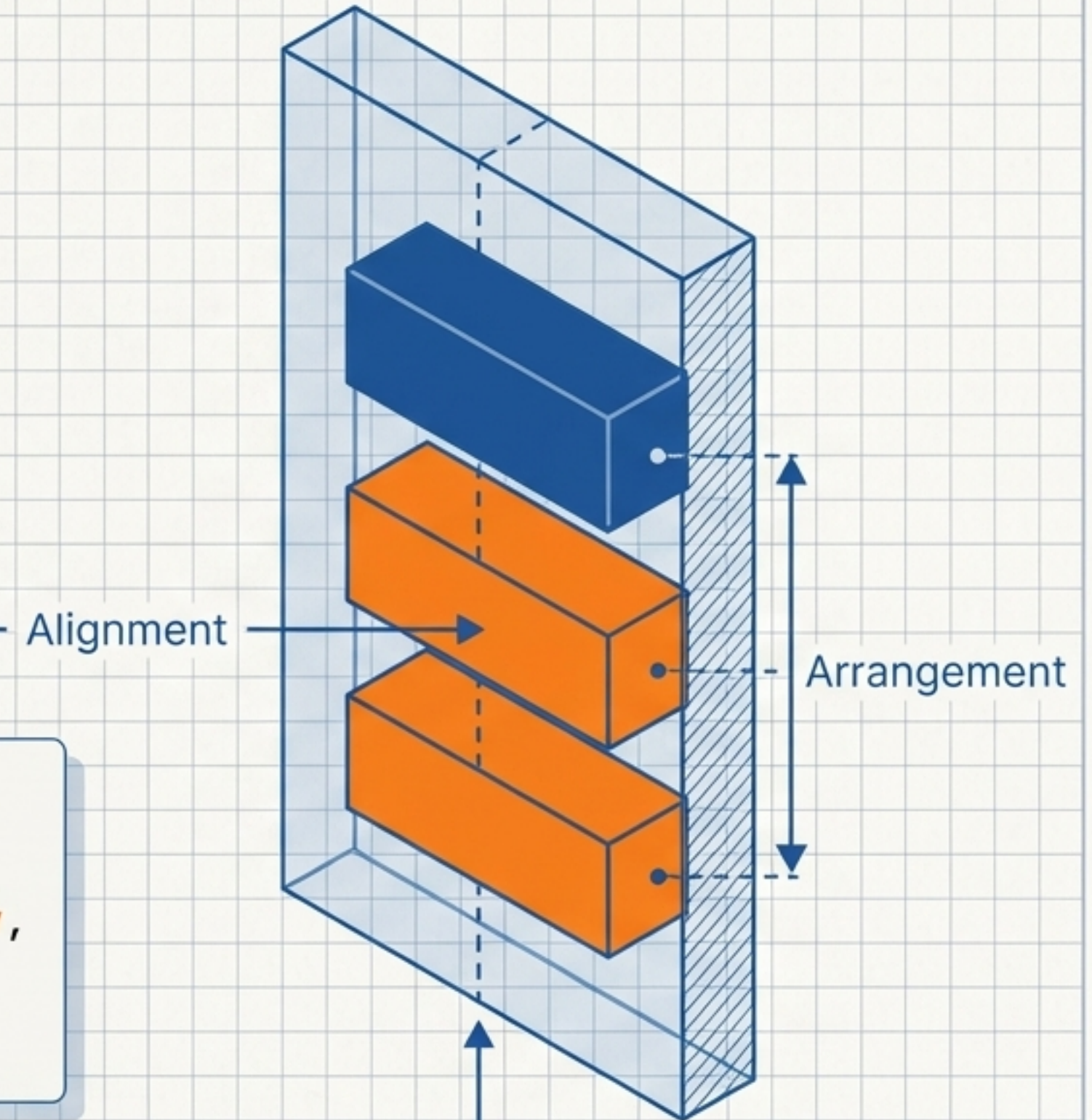
```
enum class EstadoPantalla { MENU, CONECTA_4, HUNDIR_FLOTA, INVASORES, DAMAS }  
var pantallaActual by remember { mutableStateOf(EstadoPantalla.MENU) }
```

Construyendo layouts: Apilando bloques

Column es nuestro contenedor principal. Funciona como una caja vertical que organiza los elementos uno debajo del otro, alineándolos perfectamente en el centro de la pantalla.

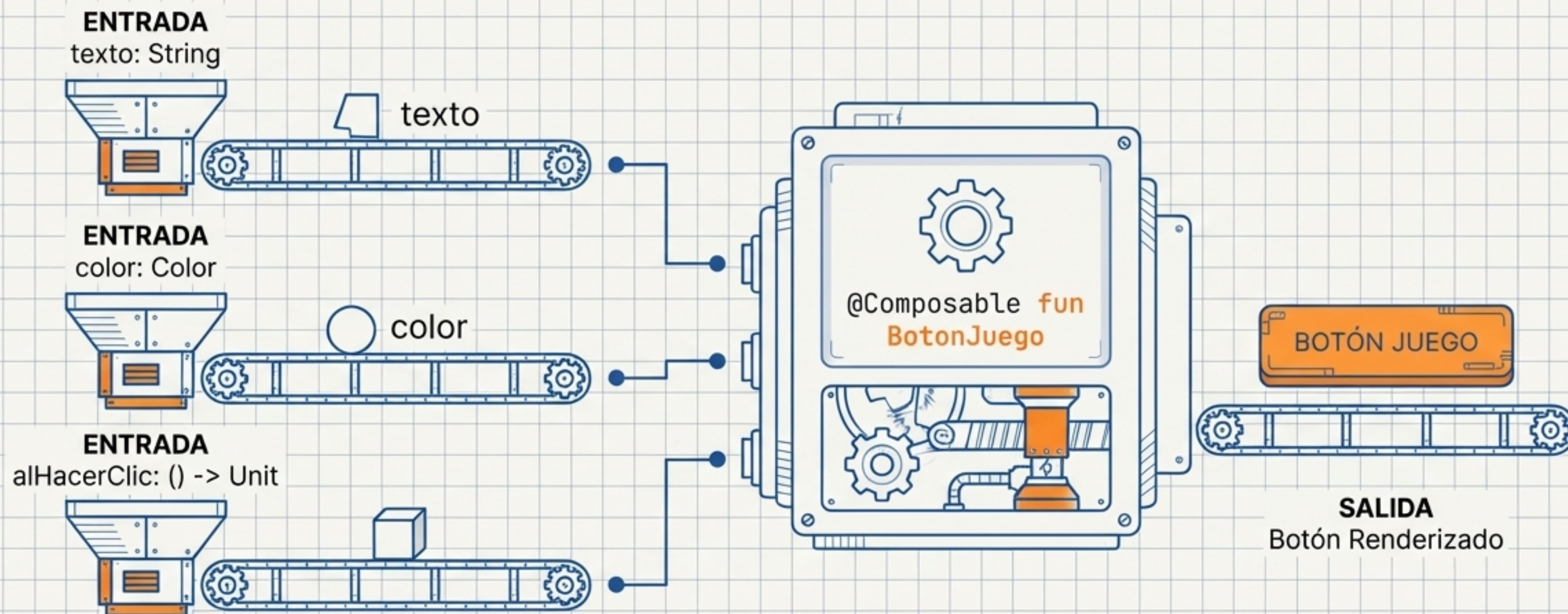
Eje X: Alignment.CenterHorizontally
Eje Y: Arrangement.Center

```
Column(  
    modifier = Modifier.fillMaxSize(),  
    horizontalAlignment = Alignment.CenterHorizontally,  
    verticalArrangement = Arrangement.Center  
)
```



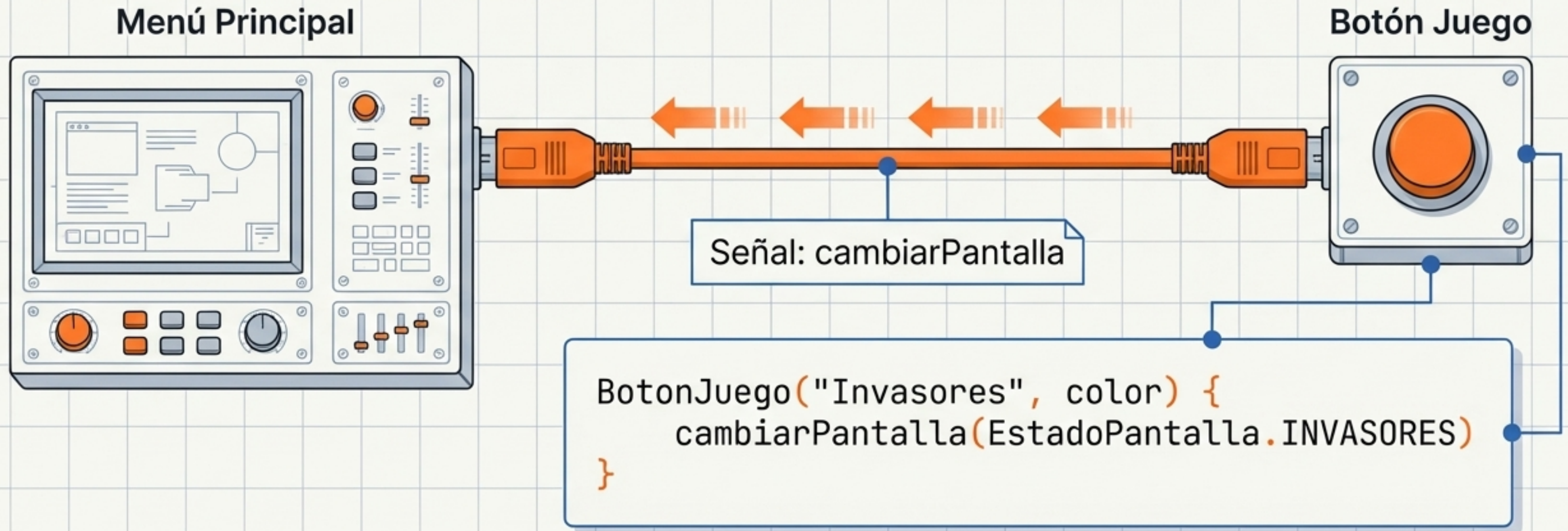
Reutilización: Creando componentes

En lugar de escribir 8 líneas de código por cada botón del menú, creamos una plantilla reutilizable. Pasamos los datos específicos como parámetros y Compose hace el resto.



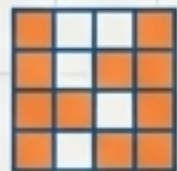
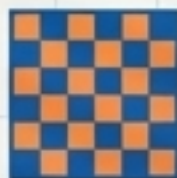
Pasando acciones como parámetros (Lambdas)

() -> Unit significa "una acción que no devuelve datos". Nos permite inyectar comportamiento. El botón no sabe qué hace al ser pulsado; solo ejecuta la acción que el menú le ha conectado a través del cable.



Los 4 Cartuchos: Un mismo patrón arquitectónico

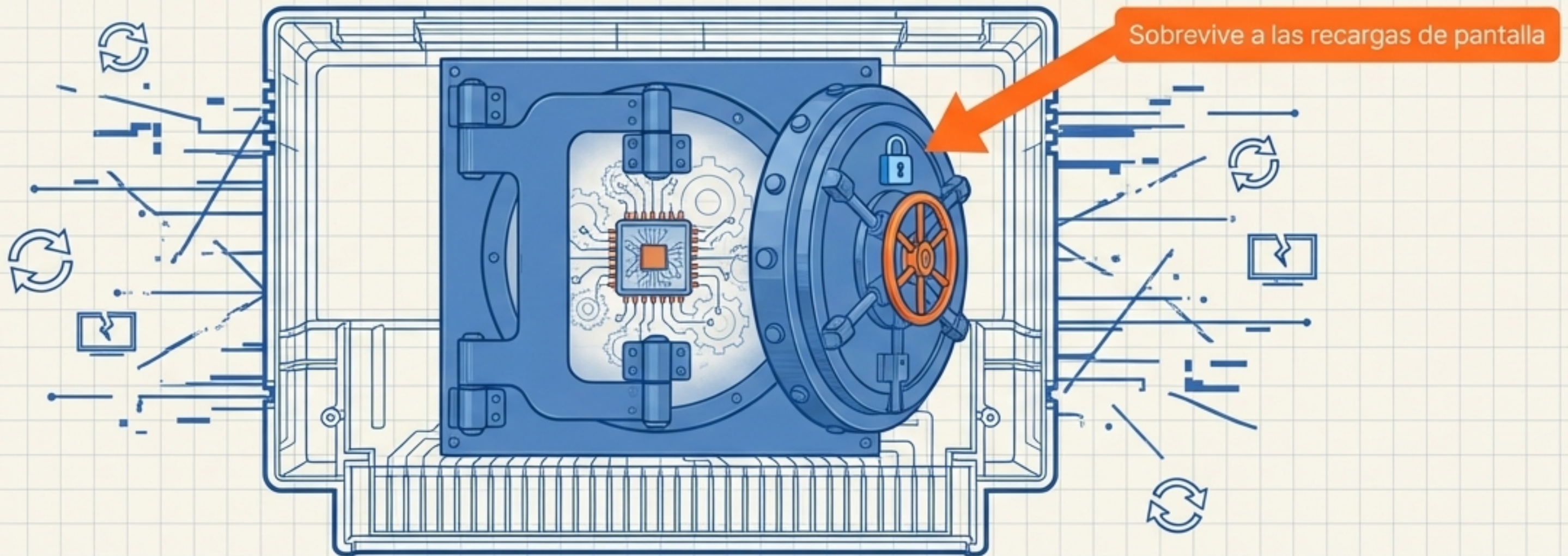
Aunque los juegos parecen distintos, bajo el capó comparten exactamente la misma estructura. Entender uno significa entenderlos todos.

Juego	Interfaz (Compose UI)	Motor Oculto (Clase Lógica)	Control de Tiempo (Corutinas)
 Conecta 4	PantallaConecta4	JuegoConecta4	Sin tiempo real
 Hundir Flota	PantallaHundirLaFlota	JuegoHundirFlota	Sin tiempo real
 Invasores	PantallaInvasores	JuegoInvasores	Bucle Continuo
 Damas	PantallaDamas	JuegoDamas	Espera Turno IA

Protegiendo el cerebro del juego

La interfaz de Compose se redibuja constantemente. Usamos remember para instanciar nuestra clase de lógica una sola vez, garantizando que no perdamos la partida cada vez que la pantalla parpadea.

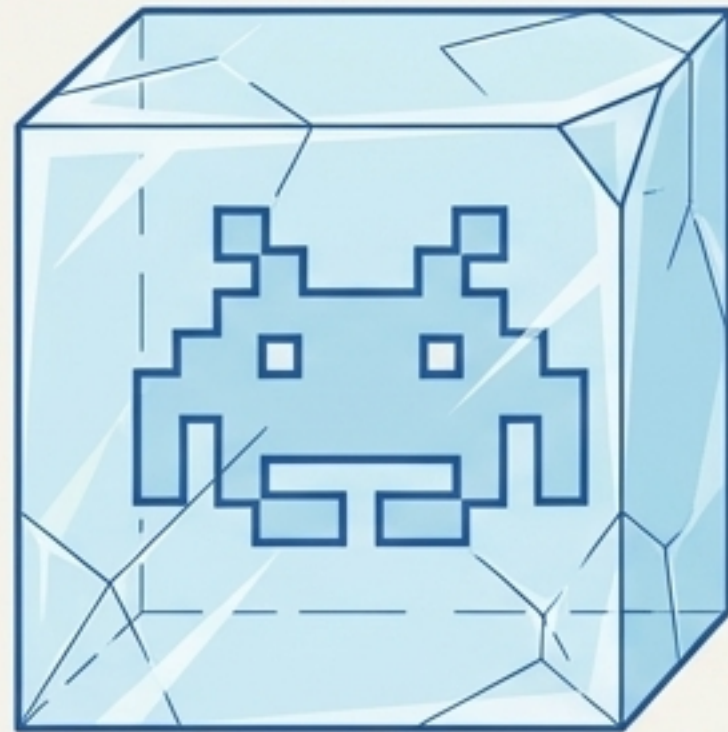
```
val motorJuego = remember { JuegoConecta4() }
```



Dando vida a lo estático con LaunchedEffect

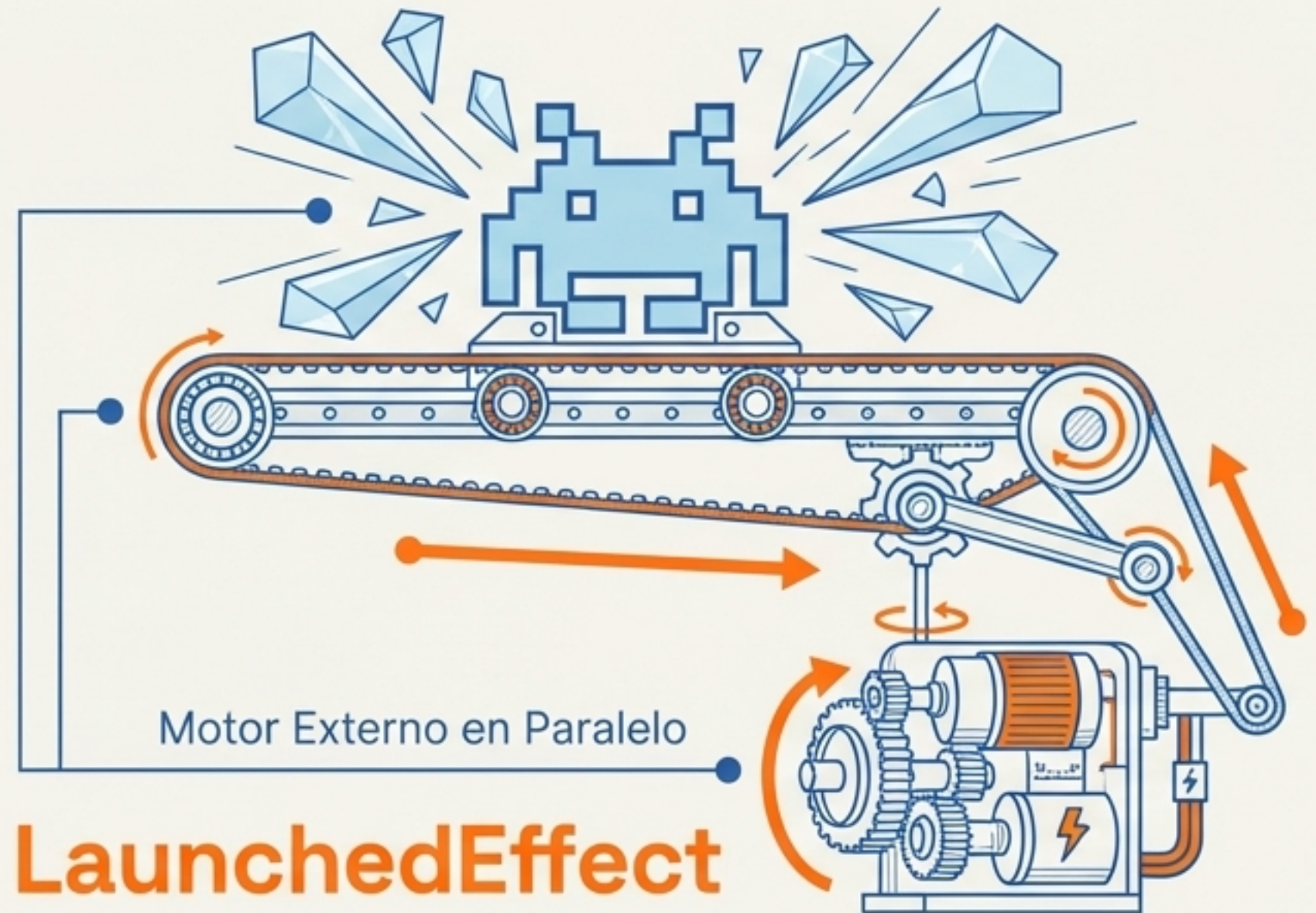
Compose por sí solo dibuja pantallas fijas. Para animar enemigos o pausar turnos, necesitamos un motor externo que corra en paralelo sin bloquear la interfaz. Las corutinas son ese motor.

Interfaz Estática



Bloqueo Estático

Corutina Activa



Motor Externo en Paralelo

LaunchedEffect

El bucle de tiempo continuo (Invasores)

Usamos un bucle infinito que pausa una fracción de segundo, actualiza las posiciones en la lógica y fuerza a la pantalla a redibujarse incrementando una variable de estado.

`while` (juegoEnMarcha)

1. Pausa



`delay`(INTERVALO)

2. Mover



`motorJuego.turno()`

3. Redibujar

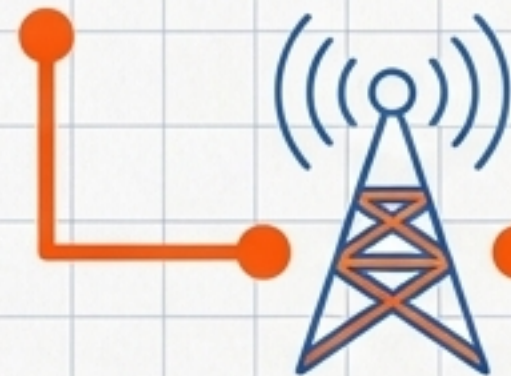
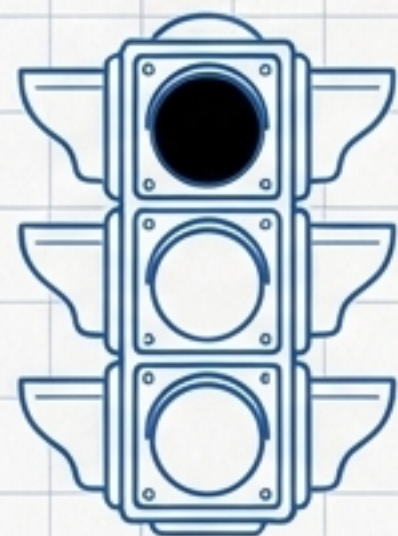


`refrescar++`

Inteligencia Artificial y turnos reactivos (Damas)

A diferencia del bucle continuo, aquí el efecto secundario solo se dispara cuando la variable `turnoActual` cambia. La pausa de 500ms simula que el ordenador está pensando antes de mover.

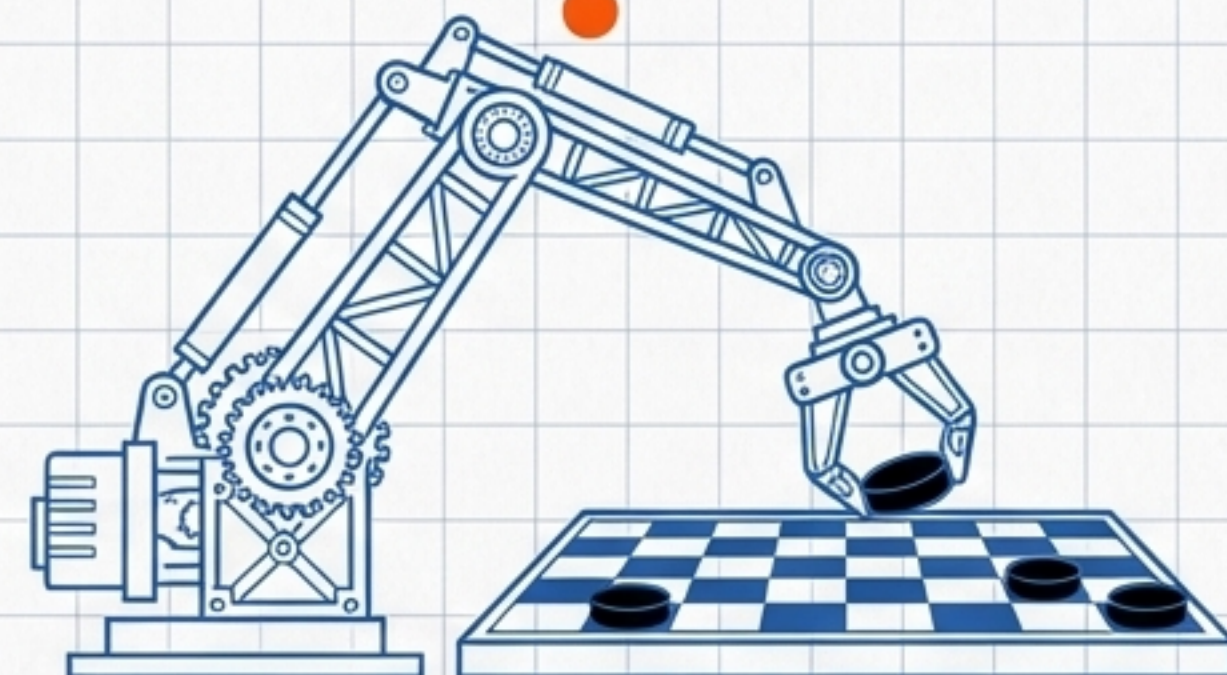
Estado: **turnoActual**
Si es NEGRO



LaunchedEffect



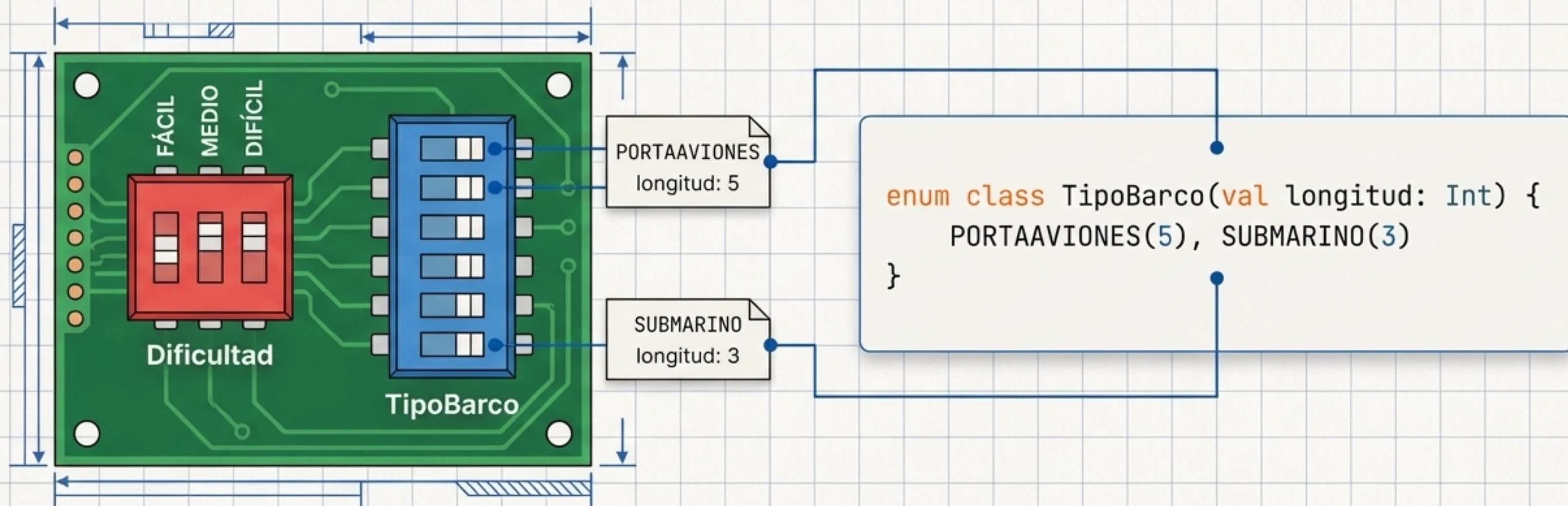
Esperar 500ms



Mover IA

Estructurando las reglas con Enum Class

Kotlin nos permite definir conjuntos cerrados y seguros de opciones. Al adjuntarles datos, como la longitud del barco, mantenemos el código limpio y libre de errores tipográficos.



Síntesis: Tu nueva caja de herramientas

Has desarmado una máquina compleja y descubierto que está construida con solo 5 piezas fundamentales.

1. Estados (**remember**)

La memoria RAM

4. Eventos (**() -> Unit**)

Los cables de señal

2. Layouts (**Column**)

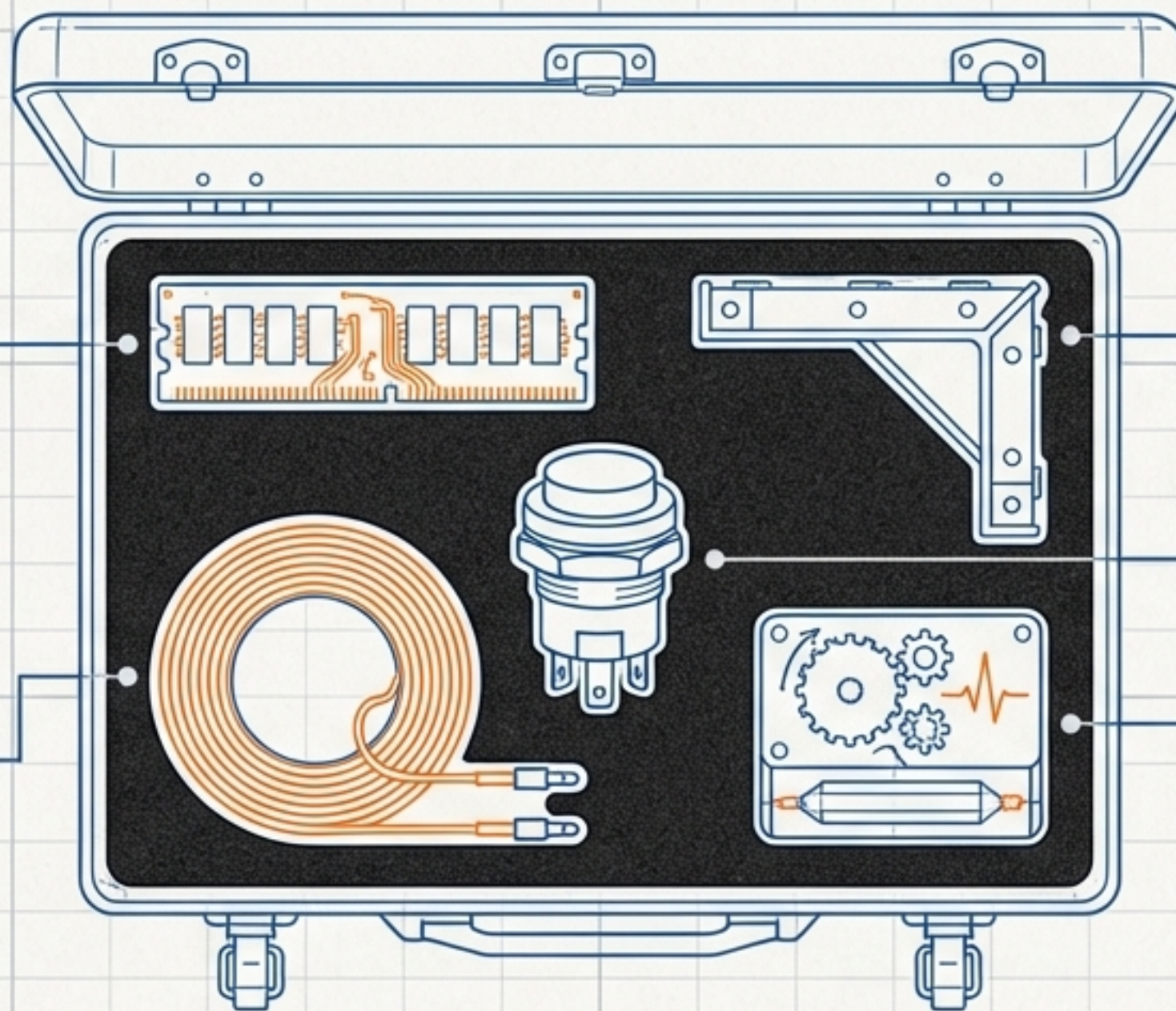
El chasis físico

3. Componentes
(**@Composable**)

Los botones reutilizables

5. Efectos
(**LaunchedEffect**)

El reloj del procesador



Nivel Completado. ¿Hacia dónde ir ahora?

Has dominado los fundamentos visuales y de estado. Para tu próxima aventura en Android, te recomendamos explorar:



Módulo Básico ✓

ViewModel

Para gestionar el estado de forma más robusta.

StateFlow / LiveData

Para observar datos en arquitecturas complejas.

Inyección de Dependencias

Para conectar motores lógicos a mayor escala.